HARDWARE VERIFICATION · A HOLISTIC GUIDE · PART III

Dynamic Verification

Testbench architecture, stimulus, UVM, assertions and regression — the simulation core of the discipline, built as one argument.

Lecture companion to Chapters 8–12.

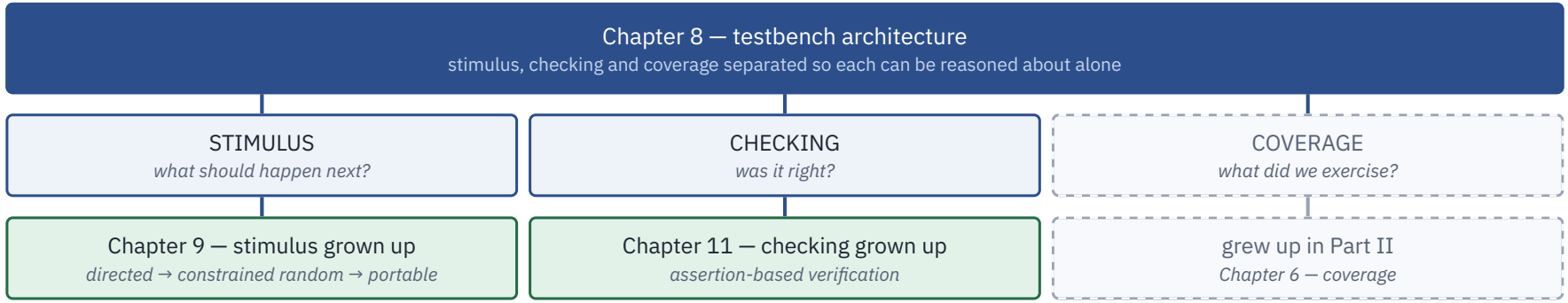
Five chapters, one claim

A testbench is an **architecture with a checking contract** — not a pile of components.

- **Why five responsibilities stay apart**, and what specific capability you delete each time you merge two of them.
- **How stimulus grew up**: from writing tests to writing rules — and the new failure mode that arrives with them, in which the stimulus looks busy and covers nothing.
- **What a standard library buys**, and the exact currency it is paid for in: indirection, spent at debug time.
- **How checking grew up**: a specification sentence, written so a tool can evaluate it, sitting on the port where it applies.
- **How to run all of it every night** without losing the one failure that matters.

SAY *Part I asked why verification is hard. Part II asked what to measure. Part III is where the instruments get built.*

Chapter 8 separates three jobs. The rest of the book grows them up.



Chapter 10 — UVM. The library that standardizes the wiring, so a component survives a change of level, of project, of company.

Chapter 12 — regression. The factory that turns commits into evidence, every night, at a cost you can defend.

SAY *Ten and twelve are the infrastructure either side of the argument: one holds the environment together, the other runs it.*

08

Testbench Architecture

Five responsibilities, not one program — and why that is not bureaucracy.

CHAPTER 8 · THE OPENING SCENARIO

It works. It has found four bugs. Then one week happens.

Six weeks into crossbar bring-up: **one file, 2,400 lines**. It drives all five manager ports, samples the four subordinate ports, computes what the memory should return, and collects coverage — in the same forever loop, over the same variables.

MONDAY — A MISMATCH NOBODY CAN READ

A read-data mismatch fires at 41 µs. The crossbar may have returned two responses in a legal order; the checker may have assumed an order the protocol never promised; the driver's back-pressure model may have drifted. **All three hypotheses share state, so no experiment separates them.**

THURSDAY — A CHECKER THAT CANNOT MOVE

The DMA team asks to reuse the AXI checking. It cannot be lifted out: the checker reads variables the driver writes, so it knows only what the driver *intended* to send — never what appeared on the wires.

ONE ROOT, AND IT IS NOT A CODING ERROR

Five distinct responsibilities were implemented as one program. Two days in a waveform viewer later, the ticket closes as *"testbench issue"* — the phrase that means **we never found out**.

SAY *Ask the room which of the two failures they have personally shipped. Both hands go up.*

Five responsibilities. Each earns its own box.

RESPONSIBILITY	ANSWERS	MUST NOT KNOW
Stimulus generation	*What* scenario should happen next?	How the protocol wiggles pins
Driving	*How* is this transaction expressed on the interface?	What the correct response is
Monitoring	What *did* appear on the interface?	What was intended
Checking	Was it *right*?	Which test is running
Coverage	What have we *exercised*?	Whether the test passed

Read the third column first: each box is defined as much by what it is denied as by what it does. And the separation is not a modern refinement — by 2001 a language built specifically for functional verification already had a data model, a test generator, a reference model, a checker split into data and temporal halves, and coverage tied back to a plan.

SAY *The structure came first; the tooling followed. Anyone who tells you this is UVM ceremony has the history backwards.*

Merge any two and you silently delete a capability

CHECKING INSIDE THE DRIVER

Destroys the oracle. The checker now compares the design against the stimulus generator's *intentions*. Every bug in which the driver itself misbehaves becomes invisible — both sides of the comparison move together.

MONITORING INSIDE THE DRIVER

Destroys reuse. If observation is a side effect of driving, you can never watch an interface you are not driving — which forecloses passive reuse at system level, and the best debug configuration there is.

COVERAGE INSIDE THE CHECKER

Destroys honesty. Sample where the verdict is computed and you sample only paths the checker reached. Holes then look like missing stimulus when they are missing *observation*.

THE RULE THAT GENERALIZES — AND THE SCENARIO THAT PROVES IT

A reviewer asks what happens if the driver mis-encodes `awsize`. With the split, the monitor reconstructs what the crossbar actually received and the read-back check fails. Without it, the driver's mistake propagates identically into both the design and the expectation, and **the test passes**. Hence: **predict from what was observed, never from what was requested.**

SAY *This is Chapter 3's common-mode error, promoted from a risk to an architectural guarantee.*

Signal, transaction, sequence — and never let observation block

SEQUENCE
scenarios: ordered, constrained collections that express intent — Chapter 9's subject

TRANSACTION
the unit of meaning: injected into the running simulation, terminated later by an observed result

SIGNAL
pins, edges, handshakes

The **transactor** converts between the bottom two: every physical-level operation of one interface encapsulated in one place. Everything above it speaks transactions; only it speaks pins.

Layer by protocol, not by pin bundle. What is a transaction at block level is not one at system level — for a network interface block, a frame; for the system that reassembles and routes, a packet.

	OPERATIONAL	OBSERVATIONAL
Shape	point-to-point	broadcast
Blocking	yes — and back-pressure is meaningful	**never**
Subscribers	one	zero, one, or many

WHY THE ASYMMETRY IS THE WHOLE POINT

A monitor must never be slowed by the things watching it, **or the act of measuring changes the timing of the measurement.** It also makes observers pluggable: coverage collector, protocol checker and scoreboard subscribe to one stream, and a fourth costs nothing structurally.

THE OBLIGATION EVERYONE VIOLATES ONCE

Every subscriber receives a handle to the *same* object, and a subscriber's write shall not modify it. A coverage collector that normalizes an address in place corrupts the scoreboard's expectation — and it will look like a design bug for as long as you let it.

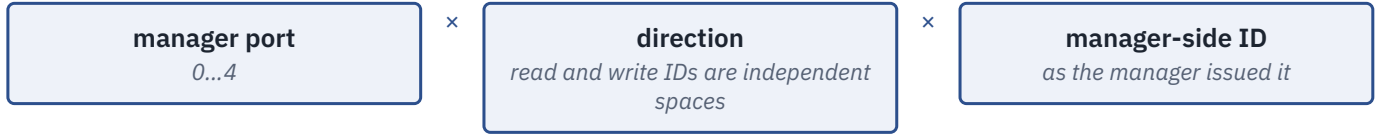
Three patterns, and a ledger you pay in ordering checks

DESIGN BEHAVIOR	STRUCTURE	LOOKUP	COST TO THE CHECKER
Preserves order	one queue	front only, O(1)	cheapest; any reorder = failure
Per-stream order only	queue per stream key	front of one queue	must **name** the stream key correctly
Reorders freely	associative index on a content key	hash, O(1) amortized	ordering is no longer checked at all

Read the last column as a ledger: **every step down buys tolerance of legal reordering and pays for it by giving up an ordering check.**

Naming the crossbar's stream key

A key acquires **one dimension per independent source of concurrency**. Here there are three: five managers issuing at once, two independent ID spaces, and per-ID ordering within one port and one direction.



WHY THE TEMPTING TWO-COORDINATE KEY FAILS

`(subordinate, ID)` merges two managers that both have ID 3 outstanding — a mismatch that *disappears on a seed change*; it merges the read and write ID spaces, so a read expectation can be popped by a write response; and it misreads the protocol, since an interconnect must **widen** the ID toward the subordinate. Five managers need $\lceil \log_2 5 \rceil = 3$ manager-identifying bits, so a **4-bit** manager ID arrives at a subordinate port as a **7-bit** one.

SAY *The constraint that decides it: a key must be computable from what the observer sees. Key on the manager side and you never model the design's ID-remap policy at all.*

Four details, each of which has cost someone a week

```
// Called by the routing PREDICTOR, fed by a monitor.
// Never by a driver. (Section 8.1's rule, in code.)
function void predict(xbar_exp_t e);
  if ($isunknown(e.id)) begin
    $error("xbar_sb: X/Z in a request ID, mgr %0d", e.mgr);
    return;              // else int'(id) files it under key 0
  end
  pending[stream_key(e.mgr, e.dir, e.id)].push_back(e);
endfunction

// Called by a MANAGER-side monitor, so what is checked is
// the crossbar's return path -- not our own responder.
head = pending[k].pop_front();
if (head.beats !== beats) begin   // !== , never !=
  n_mismatch++;
  ...
end

function int unsigned leftover();   // end of test
  foreach (pending[k]) n += pending[k].size();
endfunction
```

1 · `!==`, NOT `!=`

Logical inequality on operands containing x or z yields `1'bx`, which `if` reads as **false**. With `!=`, a single undriven beat scores as *matched* — a green result from a comparison that could not go red.

2 · THE `$ISUNKNOWN` GUARDS

`int'(id)` is a two-state cast: it collapses an unknown ID to `0` and files a corrupt transaction under a real key.

3 · `LEFTOVER()`

The interesting failure is usually **silence**. A response that never arrives produces no mismatch — only a queue that never empties.

4 · ZERO-DELAY ENTRY POINTS

Both are functions, so a checker can never back-pressure a monitor. And every record carries its manager, decoded subordinate, capture index and prediction time — the cost is four fields, the saving is the opening scenario's two days.

FACILITATION *"A silent scoreboard is a broken scoreboard." Ask what a report line with four counters — matched, orphan, mismatch, leftover — would have said on their last project.*

A predictor you wrote is not golden

THE DISTINCTION TEAMS BLUR

A **transfer function** reproduces the design's transformation so the environment can predict an output. It does not exist a priori, it is not golden by definition, and it will contain as many errors as the design itself. When model and design disagree, the first question is *which of the two misread the specification*.

1 · ALGORITHMIC

You write it from the specification. The crossbar's address decoder; the DMA's descriptor legalizer. Cheap to build, cheap to change, **never trusted absolutely**.

2 · GOLDEN EXECUTABLE

The standard ships the model, so checking collapses to comparison. A video decoder is the clean case — and the *encoder* beside it on the same die is where the species runs out.

3 · CO-SIMULATED ISS

The model runs alongside the RTL; comparison happens at **instruction retirement** — PC, registers, control registers, trap status.

Golden is not the same as complete. A standard constrains the bitstream, not the choices that produced it: two conformant encoders emit different bitstreams from the same picture and both are correct. What replaces the model is a conjunction of weaker claims — it decodes, within a distortion target *the team itself* set, without breaking buffer constraints. Each checkable; none golden.

Two conditions on co-simulation. Models need only be *functionally* accurate, not cycle-accurate — which dictates that you compare at an architectural boundary, never cycle by cycle. And **every asynchronous event must reach both sides**: notify the model of the same interrupts, or each random interrupt becomes a divergence that costs a day and is entirely an artifact of your environment.

SAY *Whatever species you build, you have written a second implementation of the specification. Every spec change is now two edits, two reviews and one argument.*

A monitor never drives — and the modport is what enforces it

```
interface axi_mon_if (input logic clk,
                      input logic rst_n);
  logic        awvalid, awready;
  logic [31:0] awaddr;
  logic [3:0]  awid;
  clocking cb @(posedge clk);
    input awvalid, awready, awaddr, awid;
  endclocking
  // Only the clocking block and reset are
  // reachable through this modport.
  modport MON (clocking cb, input rst_n);
endinterface
```

A monitor handed a `virtual axi_mon_if.MON` cannot assign to the bus at all: **the compiler rejects it, so no code review has to catch it.**

> **THE SHORTCUT VERSION OF THIS CLAIM IS FALSE**
>
> A component holding a plain `virtual axi_mon_if` handle reaches *every* member: `awvalid` and friends are ordinary `logic` variables, so `vif.awvalid = 1'b1;` compiles and drives. A clocking block on its own constrains `vif.cb.*` and **nothing else**.

Who initiates — not which direction

Signal direction is the wrong criterion. If a task *initiates* under testbench control it is stimulus; if it *waits* for a transaction the design initiated it is a monitor. So a subordinate-side responder is an agent's **driver**, however passive it feels.

And it must always be monitoring

Activate a response task late and the design finds the testbench not ready: back-pressure builds so throughput is never verified, output data spills, or the protocol is violated outright — *a protocol error that is entirely the testbench's fault.* The answer is the **autonomous monitor**: a thread started at construction, pushing into a FIFO the environment drains at its leisure.

SAY *The failure mode to name aloud: a monitor that infers what it cannot observe has become a second design — with its own bugs, and nobody reviewing it.*

The crossbar environment — and the contract written before any code

Nine agents, one per interface — **all nine carry a monitor**, including the interfaces the environment itself drives. Monitors publish; none of the subscribers can block them, and none may modify what it receives.

STEP 0 — FOUR VERDICTS, EACH WITH A NAMED OWNER

Data integrity per ordered stream and completeness → the **scoreboard**. Routing correctness → the **forward-path check**. Protocol legality on nine interfaces → the **checkers**.

AND THREE IT WILL NOT PRODUCE

Relative ordering between different transaction IDs. Arbitration fairness. Latency bounds. **Keep the negative list** — it is the half teams skip, and the half that stops a reviewer assuming a check that does not exist.

Step 5 — what this environment cannot see. The canonical crossbar bug is write-data interleaving at a subordinate port under back-pressure: the subordinate stalls `WREADY` on the first data beat and the write arbiter releases the grant mid-burst, so beats of two bursts arrive interleaved. The scoreboard is blind to it *by construction*: the violation is a relation **between** two transactions, not a property of either one's data. It was found by protocol assertions in formal, with an 11-cycle counterexample.

Take the environment you work on today

QUESTION 1

Write its **checking contract** from memory: every verdict it produces, and the component that owns each. How long did that take — and did you have to go and read code to finish it?

QUESTION 2

Now write the **negative list**: three things a reasonable reviewer might assume it checks, and it does not. Who else on your team knows that list?

QUESTION 3

Find one place where two of the five responsibilities are merged. What capability did that merge delete — the oracle, passive reuse, or honest measurement? Is the merge worth keeping?

FACILITATION *Question 2 is the one that goes quiet. Let the silence run — it is the lesson. If nobody can produce the negative list, the environment's promises live only in the head of whoever wrote it.*

09

Stimulus

Directed to random to portable — stop writing tests, start writing rules, and measure what you got.

Eleven directed tests. Every one passes. What is test 12?

The DMA testbench is three weeks old: a driver, a scoreboard, eleven directed tests. Test 1 copies 64 bytes. Test 7 does a 2D transfer with a positive row stride. Test 11 puts a destination four bytes below a 4 KB frontier, because someone read the bus rule and wanted to watch the midend split a burst.

> **THERE IS NO SHORTAGE OF CANDIDATES**
> Negative strides. Both channels at once. A row exactly one maximum burst long, starting exactly one maximum burst below a page frontier, on channel 0, while channel 1 is draining. Each is plausible. Each takes half a day to write and debug.

The list, honestly enumerated, does not have eleven more entries. **It has several thousand.**

> **AND THE DEEPER PROBLEM IS NOT SPEED**
> Writing them by hand **guarantees that the bugs you find are the ones you already imagined.**

SAY *This is the question that ends the directed era on every project that survives its first tape-out.*

Precision, and its ceiling

BRING-UP

The first stimulus a new block sees must be small enough to debug when *everything* is broken at once. A twelve-line test answers “register decode, midend, backend, scoreboard or clock?” in one waveform. A random 47-row negative-stride transfer answers nothing.

SPEC COMPLIANCE

Some stimulus is enumerated for you by an external authority, and randomizing it is a category error. A memory controller's training sequence has **exactly one legal opening move**. The value of running the industry's list is that it *is* the industry's list.

REGRESSION ANCHORS

Short, deterministic, finishes in seconds, and its failure is never ambiguous — the ideal smoke test.

WHERE IT STOPS

Manageable to completion when the total is in the **low hundreds**. A project with over a thousand identified test cases would need **more than a year** this way. A schedule claim, not a feasibility one — and quite bad enough.

TWO FAILURE MODES THAT MATTER MORE THAN THE ARITHMETIC

Bias. Hand-written stimulus is a projection of the author's model of the design; where the model is wrong, the stimulus is silent. **Decay.** A test can pass forever while ceasing to test what it was written for — enlarge a FIFO and every test that verified its full operation still succeeds while exercising only part of it.

None of this retires the technique. In 2024, random tests are still described as covering the vast majority of easy-to-detect scenarios but unable to reach complex corner cases in reasonable time, with directed tests covering the remainder. The mature position: use a directed test when a scenario is too complex to describe or too unlikely to happen randomly — and a directed sequence can be *injected* into a stream of random ones. In a mature environment, **“directed” describes a fragment of a run, not a run.**

Source: 2024 state of automated directed-test generation; directed/random division of labour.

Stop describing *a* run. Describe the set of legal runs.

Constraints are formal, unambiguous specifications of design behaviour: they define what input combinations can be applied, and when. Because they are **declarative** they read like the specification rather than like a program; because they are **executable**, the manual construction of a procedural testbench disappears.

ONE ARTIFACT, THREE HATS

The same constraint that **generates** legal traffic on one block's input can **check** legality on a neighbouring block's output — and Chapter 14 makes it a formal **assumption**. That generator/monitor duality is what makes interface constraints reusable one level up the hierarchy.

COMMITMENT 1 — YOU MUST MEASURE

You no longer know what you ran. **"If you are not using functional coverage in tandem with a random environment, you are only doing directed test cases with random stimulus filling."** The generator answers *what could happen*; only the coverage model answers *what did*.

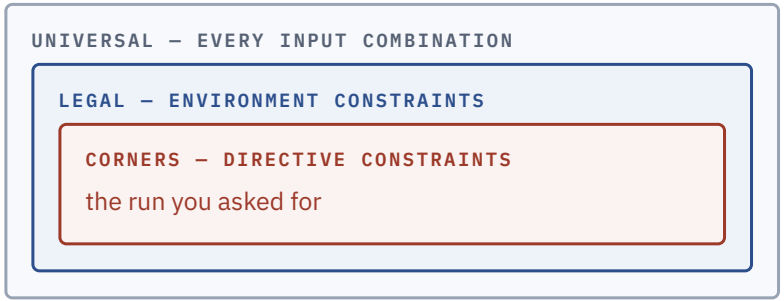
COMMITMENT 2 — THE GENERATOR IS AN ARCHITECTURE

One able to exercise the required features does not happen by accident. Design a generator that can be constrained *from the outside* rather than altering its code per test. **Every forked generator is a directed test wearing a costume.**

In the 2024 industry data, constrained-random simulation is the one simulation-based technique whose adoption is **still climbing** rather than having stabilized.

Source: Wilson Research Group 2024 IC/ASIC study; constraint semantics from the language standard and the constraints literature.

Legality is a claim about the design. Policy is a claim about your test.



The outer boundary belongs to the **design** and lives in the generator. The inner one belongs to a **test** and lives in a layer above it. Confusing the two is the origin of most bad stimulus.

```
// --- legality: the frontend rejects anything else ---
constraint c_bus_alignment {     // 64-bit data path
  src_addr[2:0]   == 3'b000;
  dst_addr[2:0]   == 3'b000;
  num_bytes[2:0]  == 3'b000;
}
// --- policy, NOT legality: an unmapped address is
// legal to PROGRAM. Disable this one in the
// error-response test.
constraint c_addressable {
  src_addr inside {[SRAM_BASE : SRAM_BASE + SRAM_SIZE - 1]};
  foreach (dst_row[i])   // a negative stride must not
    dst_row[i] inside    // walk the rows out of the map
      {[SRAM_BASE : SRAM_BASE + SRAM_SIZE - 1]};
}
// --- policy: keep the object small enough to solve fast
constraint c_solver_budget {
  num_bytes  inside {[8 : 2048]};   // <= one 256-beat burst
  num_reps   inside {[1 : 64]};
}
```

The comment on `c_addressable` is the most important line in the file. What the DMA does with an unmapped address — a decode error, reported per channel — is *a feature to verify, not stimulus to suppress*. A constraint block is the most convenient place in the language to commit that error permanently.

Label the budget honestly. `c_solver_budget`'s bounds are engineering choices about solve time, not statements about the design. Saying so lets a later test relax them without anyone wondering whether it has just generated something illegal.

SAY *Teams that stay sane require every constraint block to declare legality or policy in a comment. After a year nobody remembers which of the forty are which.*

The solver's promise is about your constraints, not your coverage model

> **WHAT THE SOLVER OWES YOU**
> Random values shall be selected to give a **uniform distribution over legal value combinations**. A strong guarantee — about the solution space *defined by your constraints*, which is not the space your coverage model measures. Nothing in the constraint model knows which edges matter.

`dist`

Weights over values and ranges. `:=` applies the weight to each element of a range; `:/` divides one weight across it. Nonzero weights **cannot cause the solver to fail** — they do not change the solution space.

The trap: values *absent* from the list, and values whose total weight is zero, are treated as **forbidden**. A `dist` that does not cover everything legal is silently an over-constraint.

`solve … before`

Changes probabilities by ordering the choice. With `s -> d == 0` over a 1-bit `s` and a 32-bit `d`:

`s` is true in exactly one of $1 + 2^{32}$ legal combinations, so `d == 0` occurs with probability $\mathbf{2/(1 + 2^{32})}$ — effectively never.

Add `solve s before d`: identical legal set, and `d == 0` now occurs with probability **slightly over 50%**.

`soft`

Preference, not requirement: discarded when it cannot hold alongside the active hard constraints. Its purpose is **layering** — a generic component supplies defaults that a specialized constraint overrides automatically.

In the DMA generator, *"most descriptors are small"* belongs in a soft constraint. *"Addresses are 8-byte aligned"* does not.

> **SHAPE THE MEASURED QUANTITY, NOT THE RAW FIELD**
> Write the distribution over **the same expression the coverpoint samples**. Shaping a raw field and hoping the interesting combination emerges is guesswork; shaping the expression that defines the bin is aim.

SAY *Weights hold absent any other constraints. Two distributions over overlapping expressions are a request — you learn whether it was granted from the coverage report.*

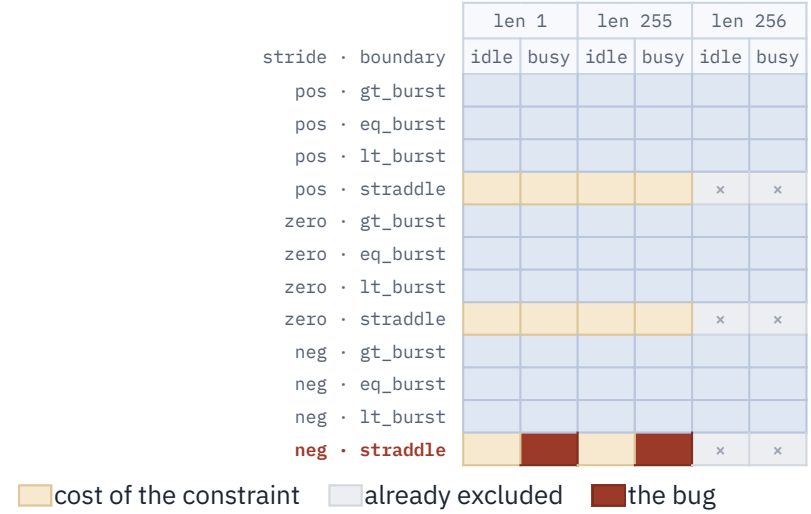
One line, added in week 4, deletes the feature under test

```
class dma_desc_overconstrained extends dma_2d_descriptor;
  // Week 4: "the AXI spec says no burst may cross a 4 KB page."
  constraint c_no_page_cross {
    foreach (dst_row[i])
      (32'(dst_row[i][11:0]) + num_bytes) <= 32'd4096;
  }
endclass
```

TRUE OF THE DESIGN'S OUTPUT. FALSE OF ITS INPUT.

The 4 KB rule governs the bursts the *backend may emit*. Splitting a request that would cross a page is precisely the **midend legalizer's job** — the feature the campaign exists to verify. The engineer has taken the design's obligation and imposed it on the stimulus, and **the tests keep passing because the design is no longer asked to do the thing it gets wrong.**

Nothing fails. Nothing warns. What the team sees is a coverage report with one clean block of cells permanently empty. The block looks like a hard corner, gets an owner and a due date, and slides.



72 cells · **18** contain `straddle` · **6** were already excluded under the generator's solver budget · the week-4 line costs the other **12** — including **both** of the bug's 2 reachable cells.

Three habits that catch it — and a diagnostic that can fail

1 · READ EMPTY CELLS AS A GENERATOR QUESTION FIRST

A gap could be a symptom of a problem in the generators, not in the design. **Structure decides which:** a whole *value* of an axis missing from every combination — a clean projection, not scattered cells — says a dimension of the stimulus is switched off.

2 · DIFF THE CONSTRAINT SET AGAINST THE COVERAGE MODEL

For each axis, find the generator variable that moves it. For `cp_boundary_dist` there is none — only a constraint pinning it.

3 · AIM, AND WATCH IT FAIL

Ask for the missing **cell** — for the relation that defines it, not for a field you hope moves it — and let the failure be the diagnosis.

```
// aim at neg x straddle: a destination row reaching past the frontier
if (!desc.randomize() with { dst_stride < 0;
                             (32'(dst_addr[11:0]) + num_bytes) > 32'd4096; })
  $fatal(1, "over-constrained: no descriptor reaches past a 4 KB frontier");
```

The second clause carries the diagnosis. Against the honest generator it is satisfiable — offset 4088 with a 2048-byte row reaches 6136. Against the over-constrained one it cannot be: `c_no_page_cross` asserts precisely the negation of what you asked for.

Aim at `dst_stride` alone and the same generator answers cheerfully with an 8-byte row, which fits inside a frame at every legal offset. *A diagnostic that cannot fail diagnoses nothing.*

AND ONE AXIS NO SHAPING CAN REACH

Nothing in a single descriptor moves `cp_other_ch`, because *"the other channel is active"* is not a property of a descriptor at all — it is a relation between two streams. **Both reachable cells of the canonical bug lie on the other-channel-active side**, so this generator, however well shaped, reaches neither.

Layering exists because constraints cannot cross generators

ATOMIC
individually constrained transactions — one descriptor

→

SCENARIO
an array of items solved at once, so the solver can relate past, current and future

→

MULTI-STREAM
one object holding a sub-descriptor per stream

Each level exists because the one below cannot express something. You cannot say *"the third descriptor reuses the second's destination"* by randomizing three descriptors in turn — and you cannot express a relation **across** streams at all, because those items live in different generators and are not inside any object one generator randomizes.

```
class contention_scenario;
  rand dma_2d_descriptor dma_ch0;  // under test
  rand dma_2d_descriptor dma_ch1;  // what cp_other_ch samples
  rand nn_job_descriptor nn;

  constraint c_channels {
    dma_ch0.channel == 1'b0;
    dma_ch1.channel == 1'b1;
  }
  constraint c_contend {        // one SRAM bank, all three
    nn.wgt_addr[4:3]      == dma_ch0.dst_addr[4:3];
    dma_ch1.dst_addr[4:3] == dma_ch0.dst_addr[4:3];
  }
  // a TEST layer, not the environment
  constraint c_worst_cell {
    nn.weight_bits     == 2;
    nn.tensor_width    == 1;
    dma_ch0.dst_stride  < 0;
  }
endclass
```

WHAT EACH CLAUSE IS DOING

`c_channels` closes the axis no single generator could reach: a descriptor on channel 1 is what makes `other_channel_active` true while the coverage model samples channel 0's transfer. `c_contend` equates bank-select bits, turning *"all three are busy"* into *"all three are contending"*.

CONSTRAINTS STATE RELATIONS, NOT SCHEDULES

Pinning channel numbers and bank bits guarantees the three descriptors **would** contend if they ran together. Making them run together is the job of the sequence that starts them — and *a scenario class without that step has aimed carefully at a cell it never fills.*

What layering runs out of at the top

Everything so far assumes a testbench that owns the interfaces. At system level that breaks: block-level sequences are not portable to firmware or post-silicon; system-level stimulus is simple directed feature tests; post-silicon failures are hard to bring back to simulation. **Three platforms, three incompatible stimulus formats, no reuse across them.**

The model

An **action** is behaviour — atomic if it maps to one operation, compound if it encapsulates a flow of others in an **activity**. Actions consume **objects** whose type fixes the scheduling, and claim **resources** from sized **pools**: two DMA channels is a pool of two, so no more than two channel-locking actions run at once.

> **WHAT DISTINGUISHES IT FROM A CONSTRAINT CLASS**
>
> **Inference.** Where intent is partial, the tool infers the additional actions needed to make it complete and valid — and infers nothing that is not required. You write *"do a 2D transfer, then run an inference job"*; the tool works out that the transfer needs a free channel and that the accelerator must have been configured first.

Honest limits

- Generic actions can be complex to create, and may need **procedural** description — the declarative model does not cover everything.
- It prefers **complete control**: a bare-metal environment with statically allocated tests — which needs a bare-metal library, and a multi-discipline team to build one.
- Test *generation* time can increase **exponentially**. The solver cost reappears at scenario scale, before simulation even starts.
- Diagnosing an over-constraint across an *inferred graph* is harder than in a class.

> **AND TWO FROM THE STANDARD ITSELF**
>
> The value concentrates in scenario *composition*, not checking. **It generates the test; the oracle remains your problem.** And realization code is per-platform work — *portable describes the model, not the effort.*

Source: Portable Test and Stimulus Standard 3.0, August 2024; industrial trial observations, 2019 — one of its six, on activity inputs and outputs, describes a 2019 toolchain and should be re-checked against what you are offered today.

Open your generator's constraint file

QUESTION 1

Take the first ten constraint blocks. For each, answer in one word: **legality** or **policy**? How many did you have to think about — and how many could a colleague answer without you?

QUESTION 2

Find a constraint that reads like a quotation from the specification. Ask the question that matters: does it restrict the **input the design must accept**, or the **output it must produce**? Only the first belongs in a generator.

QUESTION 3

Pick the coverage hole your team has been calling "a hard corner" for longest. Is it *scattered cells*, or a whole *value of one axis* missing from every combination? Which diagnosis does that shape support?

FACILITATION *Question 3 has a right answer per project and the room usually knows it within thirty seconds of looking at the shape. The point is that nobody had looked at the shape.*

10

UVM as a Methodology

Every structural decision in the library is an answer to some version of “why did that cost three weeks?”

Both had already written a working AXI4 driver

Two engineers spend a month each on a block-level testbench — one on the crossbar, one on the DMA, whose backend is one of the crossbar's five managers. Both write a driver that turns transaction objects into AXI4 handshakes. Both write a monitor that turns handshakes back into transactions. Both write something that decides whether a response was legal.

> **NEITHER CAN USE A LINE OF THE OTHER'S CODE**
> The transaction fields differ. One uses a mailbox where the other uses a queue. One prints `ERROR:`, the other `*E`. And the two testbenches end in different ways — one on a beat counter, one on a `$finish` buried inside a task.

Then integration arrives. It can be done: **roughly three weeks of rewriting.**

The interesting question is not *how do I call* `create`. It is why that costs three weeks when both engineers had already solved the problem.

The library's own motivation says the same thing in industry language: verification spans internal and external teams, and the discontinuity created by **multiple incompatible methodologies** among those teams limits productivity. The stated purpose is interoperability, and removing the cost of rewriting verification IP for each new project or tool.

SAY *Everything in this chapter is downstream of that one invoice.*

Three orders, and they get harder in order

FIRST ORDER
one environment across many testcases of one project

→

SECOND ORDER
its components inside a system-level environment

→

THIRD ORDER
those components in a different environment, for a different design

The opening scenario failed here — the order most block-level teams silently assume they get for free.

AND THE MECHANISM THAT MAKES ANY OF THEM POSSIBLE

A component must be not merely correct but **configurable**: able to exhibit the behaviour a block-level environment needs and, *without modification*, the behaviour a system-level environment needs. Configurability is not a convenience bolted onto a testbench — it is the mechanism, and most of the library exists to provide it.

The second thing it sells is vocabulary. "Make the manager agent passive at the top level" carries a structural consequence *both of you can look up in a public document*. In-house methodologies have vocabulary too — it just does not survive a new hire, a purchased component, or an external partner.

Adoption is one-sided. In the 2024 industry study it is the predominant standard for IC/ASIC testbenches and the leading standard for FPGA testbenches, with VHDL-oriented alternatives gaining traction since tracking began in 2018 and Python-based approaches newly measured.

Source: Wilson Research Group 2024 IC/ASIC and FPGA studies.

The reusable unit is one interface — because a protocol survives integration

```
virtual function void build_phase(uvm_phase phase);
  super.build_phase(phase);
  mon = axi_monitor::type_id::create("mon", this);
  if (get_is_active() == UVM_ACTIVE) begin
    seqr = axi_sequencer::type_id::create("seqr", this);
    drv  = axi_driver::type_id::create("drv", this);
  end
endfunction
```

Three load-bearing details. Sub-components are built in `build_phase`, not the constructor, because a virtual function can be overridden and a constructor cannot. They are created through `type_id::create` — which is what makes overrides possible — and the driver-to-sequencer link is made later, in `connect_phase`.

> **THE SWITCH THAT CARRIES A BLOCK ENVIRONMENT UP A LEVEL**
>
> Active mode emulates a device and drives signals; **passive mode disables driver and sequencer and leaves the monitor.** That one flag is what makes second-order reuse *mechanical rather than a rewrite*.

Then the invoice arrives

Instantiate it inside the system environment, with the real core and real memories, and all nine agents go passive — one configuration field, and it works. **What breaks is everything that assumed *we* were driving:**

- **Stimulus becomes dead code** — and any checking in the driver goes with it.
- **Configuration paths move** — silently.
- **Objections change owner**, so a test that inherited its ending now ends at time zero.
- **Block-local expectations expire**: transactions now come from firmware, so the predictor needs another source of truth.

The agent survived because it was built around an *interface*. What did not survive is the code that assumed a **role**.

Reuse is bought with indirection. Indirection is paid for in debug time.

If the environment hard-codes `axi_driver`, a test needing a different driver must edit the environment — and the environment is the artifact you were trying to reuse unmodified. The factory is the answer; the configuration database is its twin.

```
// Replace the driver of manager port 3 ONLY; the other
// eight agents keep the standard driver. Registered
// before super.build_phase() builds env.
set_inst_override_by_type("env.mgr_agent[3].drv",
                          axi_driver::get_type(),
                          axi_stall_driver::get_type());
super.build_phase(phase);
```

That test provokes the crossbar's canonical bug — write-data interleaving at a subordinate port under back-pressure — **without touching one line** of the environment, the agent, or the driver it extends. A *type* override instead would have made all nine agents stall: the choice between the two is the choice between *“this scenario”* and *“this policy”*.

NOW THE BILL

Both mechanisms replace a **compile-time reference** with a **runtime lookup**. So both defer their failures to runtime and report them somewhere other than where the mistake was made:

- A `set` whose path pattern has a typo **silently does nothing**. The component keeps its default and the failure surfaces a hundred microseconds later as a null virtual interface.
- An override registered *after* `super.build_phase` runs is simply **ignored**.
- An instance path that was correct before the environment moved one level down is now wrong, and **no tool will tell you**.

This is the honest trade. A hand-written testbench of a thousand lines is easier to debug than a UVM environment of a thousand lines; it is the ten-thousand-line case where the comparison reverses.

SAY *Most reported “UVM bugs” are configuration-path bugs: a `set` that never matched, or an override registered too late. Give the environment's configuration surface one owner, the way a module's port list has one.*

A testbench needs a state machine, and objections — not phases — end the test

`BUILD_PHASE` — TOP-DOWN

A parent decides what its children are — how many agents, active or passive — and must therefore run **before they exist**.

`CONNECT_PHASE` — BOTTOM-UP

Wiring can only happen once both endpoints exist, so the deepest, most local connections are made first.

`RUN_PHASE` — A TASK

Runs concurrently with the run-time schedule, and starts a global timeout whose expiry is fatal — the mechanism that turns a hung testbench into a *failed test* instead of a job that occupies a licence overnight.

Formalizing the steps is what lets independently developed block environments **combine** into a system-level one, and lets tests intervene at the right moment without violating the sequence. Phasing is second-order reuse made executable.

THE ASYMMETRY THAT DECIDES WHO OBJECTS

An **active manager agent has an agenda** — its reads and writes must finish — so it objects. A **reactive subordinate agent has none**, since it merely serves requests as they arrive, and should therefore *not object at all*. With no objections the run phase ends at time zero; with the responders objecting, the test never ends.

AND THE WINDOW DRAIN TIME CLOSES

Without it, the last write response is still crossing the crossbar when the final manager objection drops, and **the scoreboard never sees it** — "all objections dropped" is genuinely temporary. Declare the drain *per sub-system*, so it still applies when that environment becomes one of six inside a larger one.

SAY *Pitfall to name: drain time increased until the test passes. That is a green regression hiding a scoreboard that stopped draining.*

A mirror is not a scoreboard

THE MOST IMPORTANT SENTENCE IN THE REGISTER LAYER

The model maintains a **mirror** of what it believes each register holds — but its only information is the accesses it has seen. It can predict the content of registers the design does not update; **it cannot determine whether an updated value is correct.** Teams that forget it end up with a register model that agrees with the design about a value both of them got wrong.

```
//          parent size lsb access volatile reset has_rst rand
indiv
oe.configure(this,  1,   3,  "W1C",   1,      1'b0,  1,
0,   0);
```

Two arguments carry the weight, and both are the ones people get wrong. `volatile` declares that **the mirrored value cannot be trusted**, because the field changes in ways the model cannot observe — so comparing it against the mirror without a fresh read is *not a check*. `"W1C"` is a statement about prediction: if the real design *also* clears on read, you have modelled the wrong register and the model gets blamed for the divergence.

Front door and back door

A front-door access drives real bus cycles through the real path; a back-door access reaches the simulation constructs directly. Back-door read and write *mimic* the front-door side effects; `peek` and `poke` bypass the behaviour entirely.

THE REDUNDANCY IS THE POINT

When both directions share one path, a bug introduced on the way in is **undone on the way out** — so a reversed data bus is invisible to any write-then-read check that never leaves the front door.

THE CANONICAL PERIPHERAL BUG, IN TWO LINES

The overrun flag is never cleared if software reads the status register in the same cycle as a framing error. That is a *disagreement between the two access paths*: a front-door read then the clearing write leaves the model believing the flag is down, while a back-door `peek` shows it still up. Written as **“read front door, peek back door, compare”**, it is a two-line register test instead of a firmware lockup.

Choose a prediction mode deliberately. Once firmware running on the core writes the peripherals through the crossbar, an *implicitly*-predicted model is wrong within microseconds, because those writes never went through it. Use explicit prediction: a predictor per interface, fed by the bus monitor.

Four things sit outside the library — and all four decide whether you succeed

THE CHECKING STRATEGY

Nothing in UVM would have told you that same-ID write ordering under back-pressure was the property that mattered on the crossbar. That came from the specification and a plan.

THE QUALITY OF THE COVERAGE MODEL

It decides where a collector plugs in, not what it counts. A model with the wrong axes reaches 100% and means nothing — the accelerator's prefetcher bug escaped 100% code coverage entirely.

DEBUGGABILITY

The indirection bill, come due: what would have been compile-time errors are now runtime silence.

PERFORMANCE

Per-item allocation, dynamic dispatch and a report server on the message path are not free — invisible on a block run, highly visible in a nightly regression, or at the emulation boundary where transactor code must be synthesizable and a UVM component is not.

UVM removes most of the *accidental* cost of building a testbench and none of the *essential* difficulty — which is deciding what to check and proving you checked it.

AND WHEN NOT TO USE IT AT ALL

Small blocks with one narrow interface: a module-based testbench with bound assertions is finished before the UVM version elaborates. **Formal-first blocks**: a random generator *samples* where a proof *concludes*. **Bring-up throwaways**: not debt if you delete them; debt the day someone keeps them.

SAY *The rule in one line: build a UVM environment when the interface will be verified more than once — a second test, a second level of hierarchy, or a second project. If none of the three is true, you are buying insurance against a risk you do not carry.*

11

Assertion-Based Verification

A specification sentence, written so a tool can evaluate it, sitting on the port where it applies.

Nothing about that failure was hard to detect. It was hard to *find*.

The bug report opens with a scoreboard mismatch: a DMA channel copies 3 KB into the accelerator's weight buffer and eleven bytes land at the wrong offset. Two engineers spend **a day and a half** on it, because the scoreboard raised its hand at the end of the transfer — some **nine hundred cycles** after the damage — and everything in between must be walked backwards through a waveform.

THE CAUSE IS FOUR CYCLES WIDE

The DMA asserted `AWVALID` with an address; the subordinate was slow to accept it; an arbitration event made the DMA **withdraw the request and re-offer it with a different address** before `AWREADY` came back. The subordinate had already latched the first one.

AND THE RULE IT BROKE

Once you offer an address, you keep offering it, unchanged, until it is taken. One sentence of the bus specification. Decidable from **five signals on one interface**, **in the cycle it is broken**.

What was missing was not a smarter test. It was a statement of that rule, written so a tool could evaluate it.

SAY *Detection distance is the whole economics of this chapter: nine hundred cycles and a day and a half, versus one annotated cycle.*

Everyone nods at the sentence. Now write it.

“The payload must remain stable while VALID is asserted and READY is low.”

```
// WRONG: compares this cycle's payload against
// the PREVIOUS cycle's.
property p_aw_stable_wrong;
  @(posedge clk) disable iff (!rst_n)
  (awvalid && !awready) |->
    $stable({awid, awaddr, awlen, awsize, awburst});
endproperty
```

IT DOES NOT FAIL QUIETLY

A manager drives a fresh address in the same cycle it raises `AWVALID`. If `AWREADY` is low that cycle, the antecedent is true, `$stable` compares against the *idle cycle before*, and the property **fires on perfectly legal traffic**.

```
// RIGHT: the payload offered now must still be
// there at the next tick.
property p_aw_stable;
  @(posedge clk) disable iff (!rst_n)
  (awvalid && !awready) |=>
    awvalid &&
    $stable({awid, awaddr, awlen, awsize, awburst});
endproperty
a_aw_stable : assert property (p_aw_stable);
```

TWO EDITS, TWO DIFFERENT JOBS

The **cycle shift** repairs a known mistranslation: “remain stable” in protocol prose means *hold the same value into the next clock cycle*. The added `awvalid` is a **second rule**, not a sharpening of the first — *do not withdraw an offer before it is accepted* — and it is what catches the opening scenario.

WHY WRITING ONE IS WORTH THE TROUBLE EVEN BEFORE YOU RUN IT

A specification sentence can be vague and still feel complete; **a property cannot.** Writing one forces you to name the clock, the exact cycle relation, the condition under which the obligation starts, the reset behaviour and the polarity of every term — each a question you must answer from the specification or from whoever owns it. *An unanswered question shows up as a property you cannot finish typing.*

SAY *Folding two obligations into one property is a choice this book makes deliberately — worth knowing you have made it.*

CHAPTER 11 · ANATOMY

Four layers — and the layering matters more than the operator catalogue

STATEMENT
the four directives: assert, assume, cover, restrict

PROPERTY
claims over sequences; the workhorse is implication

SEQUENCE
patterns over time — a regular expression whose alphabet is Boolean and whose step is a clock tick

BOOLEAN
ordinary expressions — but over *sampled* values

SAMPLED VALUES ARE NOT PEDANTRY

A concurrent assertion uses the value a variable had **before any of that edge's updates**. So it cannot race the RTL it watches, cannot see a glitch on a combinational path, and **reads a waveform the way you do** — which is why a property that looks off by one cycle in a debugger usually is not.

`DISABLE IFF` — THE RESET GUARD

It discards any attempt in flight while its expression is true, ending it as **disabled: neither pass nor failure**. It is *asynchronous*, monitored at every time step. And there may be **at most one per assertion**, immediately after the clocking event — you cannot reset half a rule.

Nearly every real property carries one, because a design in reset violates almost every rule it has — and **a wall of reset-time failures is how assertion suites get switched off.**

Evaluation attempts

A fresh attempt starts at *every* tick of the leading clock, each running to its own verdict, overlapping the others. A property with a five-cycle window has up to five in flight — which is why a rule broken under sustained traffic produces a **burst** of failures rather than one.

Immediate versus concurrent

If the claim is *"this is true at this point in this code"*, it is **immediate** — procedural, like an `if`, where x or z counts as failure. If it involves a clock or spans time, it is **concurrent**, and it will be checked on every test that ever runs, whether or not anyone remembers it is there.

Three claims about one property text — and two very different engines

DIRECTIVE	IN SIMULATION	IN FORMAL VERIFICATION
`assert`	fails if violated on this trace	proved, or a counterexample produced, under all stated assumptions
`assume`	checked like an assertion: a violation means the *environment* misbehaved	**constrains the legal traces; its own correctness is not checked**
`cover`	reports and counts each match on this trace	asks whether a matching trace *exists*, and returns a witness
`restrict`	ignored	constrains traces, like `assume`

IN SIMULATION THE FIRST TWO ARE SYNONYMS. IN FORMAL THEY ARE NOT REMOTELY THE SAME.

An assumption is not checked — it **deletes traces from consideration**, and the assertions are proved only over what survives. *Write an assumption stronger than reality and you have proved something about a design that does not exist.*

THE MIRROR IMAGE IS THE WHOLE TRICK

The rule that is an `assume` when you verify the DMA in isolation is an `assert` when you verify the crossbar that feeds it — **and the same text serves both.**

Where the text sits, and who wrote it, are two questions. Assertions over *internal* signals need the micro-architecture, so the designer writes them **inline in the RTL** — comments that can fail. Assertions over *external interfaces* derive from the specification, are written by verification, and **must not touch the RTL**: `bind` exists for exactly this.

SAY *The hazard at the inline end: an assertion written by the same person, in the same hour, from the same reading of the specification as the RTL encodes the same misunderstanding and never fires.*

Three clauses, three visibly different amounts of machinery

1 · HANDSHAKE STABILITY

While a request is offered and not yet accepted, it stays offered and unchanged. **No memory:** the rule relates two adjacent cycles and the language expresses it directly.

2 · ONE BURST'S WRITE DATA AT A TIME

There is *no ID on the write-data channel*, so the obligation must be reconstructed — each burst's beats counted and reconciled against the length promised by the address that opened it. **Needs a counter and a queue.**

3 · A RESPONSE ONLY FOR WORK IN FLIGHT

Only for an ID with an outstanding address, and never more responses than addresses accepted. **Needs per-ID bookkeeping.**

Clauses two and three tip the property over into a **bound checker**: a small module carrying its own model of the interface. The escalation is normal, and it is the honest boundary of the technique — *a property that quietly assumes a scoreboard behind it is not reusable*, so the state goes in the file, in plain sight.

```
// The covers that keep the assertions honest.
c_aw_stall     : cover property (@(posedge clk)
                    disable iff (!rst_n) awvalid && !awready);
c_w_overlap    : cover property (@(posedge clk)
                    disable iff (!rst_n)
                    aw_hs && (aw_len_q.size() != 0));
c_same_id_pair : cover property (@(posedge clk)
                    disable iff (!rst_n)
                    aw_hs && (id_out[awid] != 0));

bind xbar_manager_port     xbar_port_checker u_chk (.*);
bind xbar_subordinate_port xbar_port_checker u_chk (.*);
```

READ THE COVERS AGAINST THE BINDING POINT

The five manager-side instances check the five managers. The four subordinate-side ones check **the crossbar itself**, which plays the manager role as far as a subordinate is concerned.

The canonical crossbar bug — two managers issuing writes with the *same ID through different ports*, whose data beats the crossbar then interleaves — is **a violation by neither manager**. It appears on the subordinate side. There, `c_same_id_pair` reads *a second write accepted with an ID already outstanding at this subordinate*: the bug's precondition exactly.

Measured by how much of it could ever have failed

TWO WAYS TO PASS AND PROVE NOTHING

Strict vacuity: if every subordinate accepts immediately, `awvalid && !awready` is never true and the assertion *reports a perfect record on a rule it never evaluated*.

Subtler: the assertion *does* run. If the sequence library issues one outstanding write per manager, `a_w_burst_len` evaluates every run and still never sees the interleaving case it was written for. **Six weeks at 100% pass, and the hazard untouched.**

The rule: every implication ships with a `cover` on its antecedent — written at the same time, reviewed in the same review.

When a cover reads zero the diagnosis forks two ways, both actionable: *no test drives the design into the situation*, or *the design can never reach it* — in which case the property is misformulated.

Vacuity depends on how you phrased the rule. `ok |-> !err` and `err |-> !ok` are logically equivalent and pass vacuously under *opposite* conditions. Make the antecedent the rare, constructed condition — then a non-vacuous pass is evidence.

THE MUTATION QUESTION, FOR EVERY ASSERTION IN A REVIEW

Name one change to the RTL that this assertion catches and **code coverage does not**. An assertion with no answer is not a check — it is a comment with a runtime cost.

```
property p_no_4kb_crossing;
  @(posedge clk) disable iff (!rst_n)
  (awvalid && awready && (awburst == 2'b01)) |->
    (16'(awaddr[11:0]) +
     ((16'(awlen) + 16'd1) << awsize)) <= 16'd4096;
endproperty
a_no_4kb_crossing : assert property (p_no_4kb_crossing);
```

The same text, two engines. In simulation this property is *sampled* — silent about the bursts the tests did not generate. Chapter 14 hands the identical text to a proof engine: *no legal input sequence whatsoever* can produce a crossing burst, or here is one that does. **Nothing in the property changes; the question asked of it does.**

Generated properties: 89% of 56 correct syntactically and functionally, one serial-bus design, after humans removed unmappable generations (2024) — scored by a criterion this slide has just argued is not evidence.

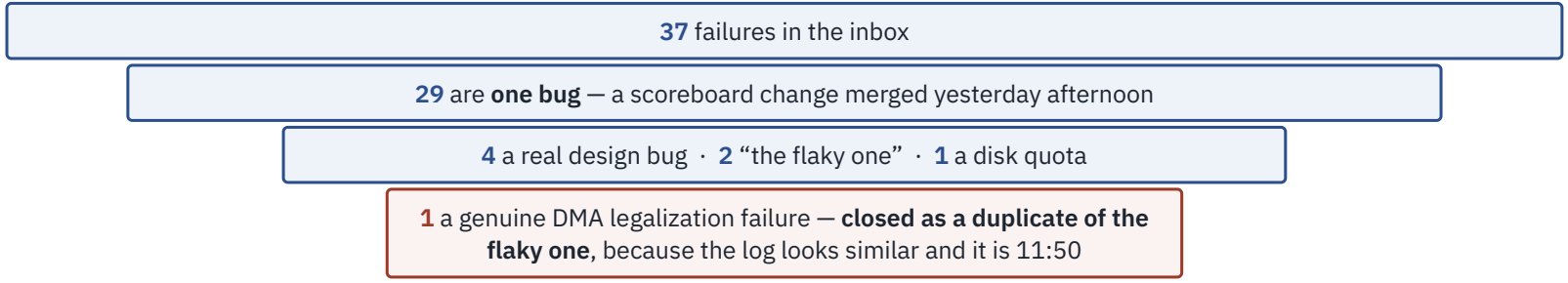
12

Regression Engineering

The machine that turns commits into evidence — and the morning it destroys.

Thursday, 07:40. Nine hundred runs, 863 passed, 37 failed.

Three engineers spend the morning on those 37. By noon the arithmetic is in.



> **NOTHING IN THAT MORNING WAS A VERIFICATION-TECHNIQUE PROBLEM**
>
> The scoreboard was right, the stimulus was right, the coverage model was right. **What failed was the factory.** One fact was reported twenty-nine times; a real find was destroyed by a signature collision with a defect the team had learned to ignore; and the loop consumed most of a working day of three engineers.

SAY *Chapter 7 taught you to read the numbers a regression produces. This chapter builds the thing that produces them.*

CHAPTER 12 · TIERS

A tier is not a schedule. It is an admission policy with a runtime budget.

TIER	TRIGGER	WALL-CLOCK BUDGET	ADMISSION RULE
Smoke	every commit / merge request	minutes	proves the build is alive; fixed seeds; bounded runtime; **zero** known failures
Nightly	scheduled, daily	one night	**discriminates:** fails when the design or environment regresses
Weekend	scheduled, weekly	one weekend	everything long, deep-seeded, or configuration-crossed
Campaign	on demand	none — bounded by its written purpose	purpose-built runs that belong on no clock

Smoke earns its place by speed and silence. A smoke tier that randomizes is one that sometimes lies; a gate with a permanent red light in it is decoration.

Nightly earns its place by discrimination. A test belongs there if a *plausible regression* would make it fail — not because it is important.

A campaign is launched for a reason, owned by someone, produces a written result, then stops. Teams keep reinventing it without a name.

> THE BUDGET IS A DESIGN CONSTRAINT, DEFENDED LIKE A TIMING CONSTRAINT
>
> Twelve minutes for lint, clock-domain checks and a 60-run smoke suite. When that grew to fifteen, **two tests moved to the daily tier rather than the slip being accepted** — because a gate a human will not wait for is a gate that gets bypassed.

Source: hardware CI tiering, 2026 industrial report — 4,536 pipelines in a three-week window, successful durations from 6.4 minutes to 2.2 days for the largest.

A constrained-random test is not one experiment. It is a family indexed by seed.

ONE SEED PROVES NOTHING MUCH

The default seed is the commonest failure of discipline in constrained-random verification: engineers keep using it until the simulation runs error-free and consider the job done — **but the same seed generates the same input sequence.** A test that passes on seed 1 has established one path through its own constraint space.

SEED BUDGETS ARE AN ALLOCATION PROBLEM, NOT A CONSTANT

A descriptor test spanning four axes has a large space to sample; a framing test with three legal configurations does not, and its seventh seed buys almost nothing. **Allocate proportionally to** *unclosed* **coverage** — groups closed and stable get one seed as a change detector.

And run one fixed seed nightly on a handful of tests. Fresh seeds measure the design; the fixed seed measures the machine — because if its result moves while its manifest does not, something outside the manifest did.

```
run_id:        nightly-20260312-0417
test:          dma_2d_stride_stress
seed:          3417755291
tier:          nightly
rtl_rev:       9f21ac4          # design repo, tagged 'regress'
tb_rev:        c07e13b          # verification repo
tool:          sim-2025.3-p2    # patch level, not just major
config:        DMA_CH=2, SRAM_KB=512, XBAR=5x4
host:          farm-node-118 (linux-x86_64, 64 GB)
result:        FAIL
signature:     SCB_DATA_MISMATCH:dma_scoreboard:beat_compare
cpu_seconds:   641
```

Every field is there because somebody once lost a day without it. The `tool` line is worth arguing about: the *major* version is common practice and is not enough. `signature` makes triage possible; `cpu_seconds` makes the economics possible.

SAY *Reproducibility must be easy, not merely possible. An incomplete manifest is a defect in its own right.*

Turning *failures* into *facts* — a data-reduction problem before it is a debugging problem

```
signature := <ERROR_CLASS>:<COMPONENT>:<CHECK>
             from the FIRST error line, with these stripped:
               - timestamps, simulation time, run ids, seeds
               - hierarchical paths below the component boundary
               - numeric operands (addresses, IDs, data values)

example:     SCB_DATA_MISMATCH:dma_scoreboard:beat_compare

counter-ex:  UVM_ERROR @ 12480ns [dma_sb]
             exp=32'hDEAD_BEEF got=32'hDEAD_BEE0
             (over-specific: every seed a distinct signature)
```

Built from the **first** error, since later errors are usually consequences. The seed is stripped from the signature and kept in the *manifest*: a dedup key carrying the seed makes every failure unique — the over-specific pathology by construction.

BOTH DIRECTIONS ARE EXPENSIVE

Over-generic buckets map many bugs into one and *eventually swallow a real find* — the opening scenario. **Over-specific** buckets destroy the reduction: 37 failures in 34 buckets is no better than 37 failures. *A signature that never splits is not a signature.*

Then classify, three ways

Design bug · testbench bug · environment failure — and the ratio is itself a health metric. Testbench bugs are not noise: the environment is a design too. Environment failures are the only category that may be discarded without debugging — *and even they must be counted*, because a rising count is an infrastructure problem masquerading as flakiness.

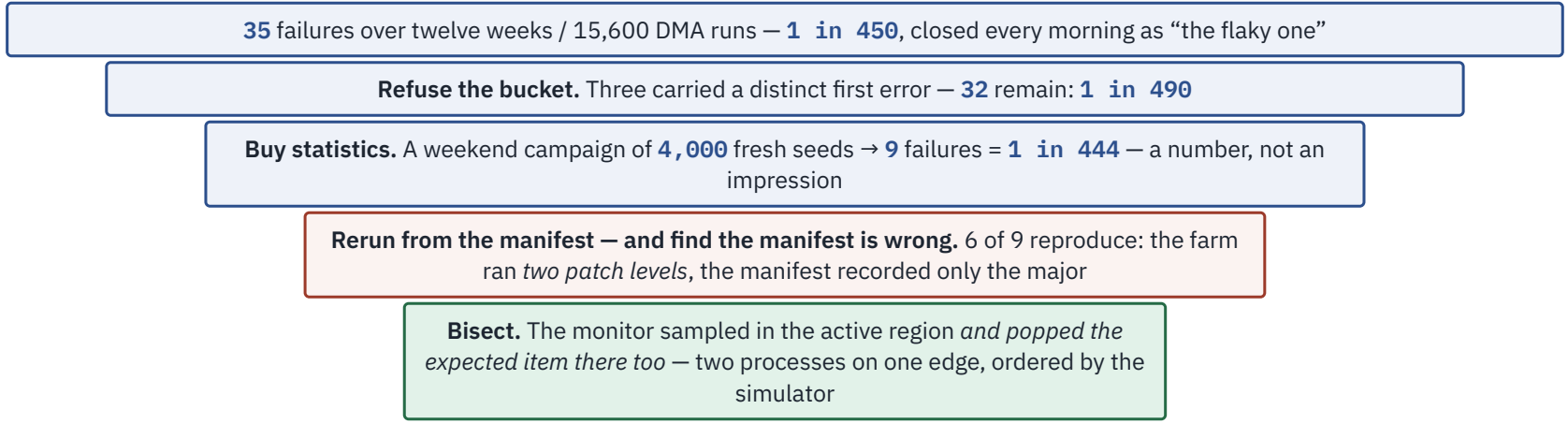
And do not debug the same thing twice

Track **errors re-found**. A suite spending 40% of its night re-discovering three known-open bugs spends 40% of its night *learning nothing*.

SAY *The organizational counterpart is a triage rota: one engineer per week who does not debug but classifies, deduplicates and assigns. Debugging and triage compete for the same person, and triage always loses — because debugging is more interesting.*

A flaky test is more expensive than a failing one

A failing test carries *information*. A flaky test carries an *instruction*, and it is **ignore red**. The opening scenario's lost bug was not carelessness — it was a team behaving rationally on the training data **its own regression gave it**.



TWO CHANGES, EACH CURING ONE HALF

Sample through a **clocking block**, so what the monitor captures no longer depends on what the design does at that edge. And the monitor **stops popping**: it enqueues, the scoreboard compares. *The clocking block is the seductive one* — it defines **when** a signal is sampled, not the order of two processes.

PROVE THE FIX HONESTLY

Re-running the 4,000-seed campaign produced **zero failures**. State what that establishes and no more: zero events in 4,000 trials bounds the true rate **below roughly 1 in 1,300 with 95% confidence.** It does not prove zero — and the difference between “bounded below 1 in 1,300” and “fixed” is the difference between an engineering claim and a hope.

What to run when you cannot run everything

The budget is not a policy but a *physical quantity*. Adding cores buys throughput, not evidence: large regressions consist of redundant simulations that do not improve coverage while still consuming resources and lengthening turnaround.

A night that no longer contains the suite

The flagship's nightly *candidate* list is **6,400 runs = 4,409 CPU-hours**. On 240 concurrent licences that is **18.4 hours** of wall clock. The night is **10**. Three decisions, in this order:

DECISION	CPU-H	WALL
candidate list	4,409	18.4 h
1 · re-tier system runs to the weekend	3,359	14.0 h
2 · rank the random block (4,800 → 1,150)	1,534	6.4 h
3 · prune the directed set	**1,417**	**5.9 h**

> **THE LOAD-BEARING DECISION LOOKS LIKE A DETAIL**
>
> **The nightly tier is ranked and the weekend tier is not.** A ranked regression reproduces known coverage and *explores nothing*, so a project whose **only** regression is ranked stops finding bugs while its dashboard stays green — the flat-curve pathology of Chapter 7, *manufactured by the regression itself*.

Ranking buys the night; the unranked weekend buys back the exploration. And the ranking is regenerated whenever the design moves significantly, so it stays a compression of the current design rather than a fossil of an old one.

Select, don't just rank. Filtering tests *before* simulating them, on the hypothesis that dissimilar tests hit dissimilar coverage, is the complementary technique: ranking buys speed at the price of exploration, novelty selection tries to buy speed *from* it.

Calibration: a signal-processing unit needed six months of simulation of nearly two million constrained-random tests on almost a thousand machines, about two hours per test (2023); novelty selection saved 49.37% of simulation to reach 99.5% coverage (2023). Ranking compressed 5,124 runs to 1,204 — a factor of 4.25 — at full coverage regain (2022 results). Distributing 1,738 tests across an eight-board emulation cluster cut 5.2 hours to 53 minutes: a 6× speed-up, not 8× (2026).

Your last nightly regression

QUESTION 1

How many **distinct facts** did it report — not how many failures? If you cannot answer without opening logs, your signatures are doing no reduction.

QUESTION 2

Name your team's flaky test. Everyone has one and everyone knows its name. **How long has it been red, and what has it taught the team to do with red things?**

QUESTION 3

If your nightly suite doubled tomorrow, which of the three levers would you pull first — re-tier, rank, or prune? What would you lose, and how would you know you had lost it?

FACILITATION *Question 2 is the one to sit with. Ask whether anyone has ever closed a real bug as a duplicate of it. In a room of practising verification engineers, someone always has — and saying so out loud is the point of the exercise.*

PART III · WHERE TO GO NEXT

An architecture with a checking contract

The through-line, restated. Chapter 8 separated stimulus, checking and coverage so each could be reasoned about alone. Chapter 9 grew stimulus up; Chapter 11 grew checking up; coverage grew up in Part II. Chapters 10 and 12 are the infrastructure either side.

SEVEN SENTENCES WORTH LEAVING WITH

- Predict from what was *observed*, never from what was requested.
- A stream key must be computable from what the observer sees.
- A predictor you wrote is not golden.
- Legality is a claim about the design; policy is a claim about your test. Label every constraint.
- Over-constraining stays satisfiable, and silently deletes the feature under test.
- An assertion that never triggers proves nothing; its antecedent's cover is the evidence.
- Flakiness teaches a team to ignore red — which is why it costs more than failure.

The chapters behind these slides

Chapter 8	Testbench Architecture
Chapter 9	Stimulus: Directed to Random to Portable
Chapter 10	UVM as a Methodology
Chapter 11	Assertion-Based Verification
Chapter 12	Regression Engineering

The scoreboard class, the bound checker and the constraint files sketched here appear there in full.

WHAT PART III DELIBERATELY LEFT OPEN

Every assertion on these slides was *sampled*. **Chapter 14** hands the same text to a proof engine and asks a different question. **Chapter 17** takes the transaction-level layering of Chapter 8 and moves the environment onto an emulator. And **Chapter 19** asks what survives at gate level.

SAY *If the room remembers one sentence: a testbench is an architecture with a checking contract, not a pile of components.*